Ders 6 Genişletilmiş Notu: Uygulama: Kitap-Raf-Kitaplık, Static Üyeler, Bileşim ve Birleştirme İlişkileri

Ders 6 Genişletilmiş Notu: Uygulama: Kitap-Raf-Kitaplık, Static Üyeler, Bileşim ve Birleştirme İlişkileri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Uygulamanın Amacı ve Soru
- 2. Kitap Sınıfı
- 3. Raf Sınıfı
- 4. Kitaplik Sınıfı
- 5. Kitaplığın Testi
- 6. Tasarım Tartışması: Hangi Metot Hangi Sınıfa?
- 7. Bileşim ve Birleştirme
- 8. Nesneler Üzerinde Yer Değiştirme Sorusu ve Sıradaki Konular

1. Uygulamanın Amacı ve Soru

Bu oturum bir uygulama saatidir. Kodu Toprak Hoca canlı yazdı. Dersin hocası Ahmet Elbir de oturumun bir kısmında araya girerek açıklamalar yaptı. Uygulama saatlerinin mantığı şudur: bir problem anlatılır, sonra birlikte çözülür.

Toprak Hoca, algoritmaları bilerek karmaşık tutmayacaklarını söyledi. Buradaki soru nesneye yönelik programlamanın ne olduğu ve kavramlarının neler olduğudur. Algoritma basit kalırsa kavramlar daha iyi anlaşılır.

Ahmet Hoca da aynı konuların YouTube'da çok sayıda videoda bulunabileceğini, bu saatlerin değerinin sorulan sorulardan ve özel olarak seçilmiş sorulardan geldiğini belirtti.

> **Hocanın vurgusu:** Anlamadığınız her şeyi sorun. Çözülen sorular belirli kavramları kavratmak için seçiliyor; fayda ancak sorarak ortaya çıkar.

1.1 Problem

Basit bir kitaplık uygulaması yazılacak. Üç kavram vardır: kitaplar, raflar ve kitaplık. Kitabın adı, yazarı ve ISBN'i vardır. Ayrıca toplam kitap sayısı saklanmak isteniyor. Kitaplar rafa eklenebilecek, silinebilecek ve aranabilecek.

Soru UML diyagramıyla verilmiştir. Kod NetBeans'te bu diyagrama bakılarak yazıldı. Yeni proje açıldıktan sonra hoca projeyi F6 ile çalıştırıp denedi.

1.2 UML Diyagramı

Toprak Hoca önce işaretleri hatırlattı: başındaki eksi (-) private, artı (+) public demektir. Private bir değişkene doğrudan erişilemez, metotlar üzerinden erişilir.

Diyagramın içeriği düz metin olarak şöyledir:

- `Kitap`
  - Alanlar: `- ad: String`, `- yazar: String`, `- ISBN: String`, `- kitapSayisi: <static> int`
  - Metotlar: `Kitap(String, String, String)`, `getAd()`, `getYazar()`, `getISBN()`, `getKitapSayisi(): int`, `setAd(String)`, `setYazar(String)`, `toString()`
- `Raf`
  - Alan: `- kitaplar: Kitap[]`
  - Metotlar: `Raf()`, `kitapEkle(Kitap)`, `kitapSilSiraNo(int)`, `kitapSil(Kitap)`, `rafGoster()`, `getKitaplar(): Kitap[]`
- `Kitaplik`
  - Alan: `- raf: Raf[]`
  - Metotlar: `Kitaplik(int)`, `kitapEkle(Kitap, int)`, `kitapSilSiraNo(int, int)`, `kitapSil(int, Kitap)`, `kitapAra(String): Kitap`, `gosterRaf(int)`, `tumRaflariGoster()`

Sınıflar arasındaki ilişkiler şunlardır:

1. `Kitaplik`, dolu elmasla `Raf`'a bağlıdır.
2. `Raf`, boş elmasla `Kitap`'a bağlıdır.
3. `Test`, kesikli okla diğer sınıfları kullanır.

Hoca elmasların anlamını dersin sonuna bıraktı; bu konu 7. bölümdedir.

> **Ders sonrası not:** Kısa nota göre soru dosyasındaki UML'de `toString(): void` yazıyor; doğrusu `+ toString(): String`. `public void toString()` yazılınca derleme hatası alınır: "return type void is not compatible with String" (javac ile denendi). Kısa nota göre diyagramda statik üye `<static>` diye gösterilmiş; standart UML gösteriminde statik üyelerin altı büyük olasılıkla çizilir.

2. Kitap Sınıfı

Hoca ilk olarak `Kitap` sınıfını yazmayı önerdi, çünkü en basit olanı ve diğerlerinden bağımsız olanı budur.

2.1 Alanlar: private, final, static

Önce üç alan yazıldı: `private String ad`, `private String yazar` ve `ISBN`.

`ISBN` için hoca UML'deki yazılışa dikkat çekti ve bu alanı `final` yaptı, değiştirilemesin diye. `final` alan başlangıçta oluştuktan sonra hep aynı kalmalıdır. Bir kitap oluştuktan sonra ISBN'ini "şudur" diye değiştiremeyiz; ama değerini okuyabilmemiz gerekir.

> **Ders sonrası not:** Büyük harfli ad tek başına bir alanı `final` yapmaz. `private String ISBN;` alanı ve ona değer atayan bir setter hatasız derlenir; `final` yazılınca aynı setter "cannot assign a value to final variable ISBN" hatası verir (javac ile denendi).

Son alan, tüm kitapların sayısını tutan statik değişkendir: `private static int kitapSayisi`. Hoca, UML'de diğer alanlarda `static` yazmadığını, yalnız bunda yazdığını gösterdi; bir fark olması gerektiğini söyledi.

2.2 Constructor, Getter ve Setter

Constructor'ı elle yazmak yerine NetBeans'in Insert Code özelliğiyle otomatik ürettirdi. Constructor üretilince `ISBN` satırındaki uyarı kayboldu, çünkü `final` alan artık constructor'da ilklendiriliyor.

Getter ve setter'lar da aynı yolla üretildi. Burada dikkat çeken bir ayrıntı vardı: `ISBN` için setter gelmedi. Sebep alanın `final` olmasıdır. NetBeans'in listesinde yalnız getter çıktı ve getter eklendi.

`kitapSayisi` için de setter yazılmadı. Bu değer dışarıdan atanmayacak, her kitap oluşturulduğunda artacak. Bunun için constructor'ın içine `kitapSayisi++` eklendi.

2.3 Statik Metot: isbnNedir

UML'de olmayan bir metot daha eklendi: ISBN'in ne olduğunu açıklayan bir metin döndüren `public static String isbnNedir()`.

Hoca ardından sınıfı C'deki yapılara benzetti: belirli özellikleri bir araya getiren, ona çok benzeyen bir şey.

2.4 Sınıf Adı ile Dosya Adı

Hoca, sınıf adıyla Java dosyasının adının aynı olmak zorunda olduğunu söyledi. Örneğin dosyadaki sınıfın adını `test` yaparsak hata verir. Adı değiştirmek için proje ağacında dosyaya gelinir ve Refactor üzerinden yeniden adlandırılır. Ana sınıf bu yolla `Test` yapıldı.

> **Ders sonrası not:** Hoca kuralı bütün sınıflar için söyledi; zorunluluk `public` sınıflar içindir. `Yanlis.java` içinde `public class Test2` yazılınca "class Test2 is public, should be declared in a file named Test2.java" hatası alınır. `public` olmayan `class Test3` aynı durumda derlenir (javac ile denendi).

2.5 İlk Test ve toString

Test sınıfında bir kitap oluşturuldu: adı `Kitap1`, yazarı `Yazar1`, numarası `1234`. Kitap yazdırılınca anlamlı olmayan bir çıktı geldi. Hoca bunun, Java'nın nesneleri ayırt etmek için kendisinin ürettiği bir şey olduğunu söyledi.

Çıktının anlamlı olması için sınıfa `toString` eklendi. `toString`'de yazılan şey, nesneyi yazdırma metoduna verdiğimizde çıkan şeydir. Kitabın bütün özellikleri yazdırıldı. Tekrar çalıştırınca kitabın adı, yazarı ve numarası göründü.

> **Ders sonrası not:** `toString` yokken çıkan metin, sınıf adı, `@` ve onaltılık bir değerden oluşur: bir denemede `V$K@1dbd16a6` görüldü, onaltılık kısım çalıştırmadan çalıştırmaya değişebilir (javac ile denendi).

Sonra `kitap1.setAd("Kitap2")` çağrıldı. `setAd` yalnız adı değiştirdiği için yazar aynı kaldı. Setter ile değer değiştirmenin, getter ile değer almanın amacı budur. Çıktı (javac ile denendi):

```
Kitap(Ad=Kitap1, Yazar=Yazar1, ISBN=1234)
Kitap(Ad=Kitap2, Yazar=Yazar1, ISBN=1234)
```

2.6 Statik Metodu Çağırmak

Hoca statik metodu şöyle tanımladı: hiçbir nesne oluşturmadan çağırabileceğimiz bir metot. `isbnNedir` doğrudan sınıfın adıyla çağrılır: `Kitap.isbnNedir()`. Bir nesne üzerinden de çağrılabilir: `kitap1.isbnNedir()`. İkisi aynı metni yazdırır.

Buna karşılık `getAd` gibi diğer metotlar sınıf adıyla çağrılamaz. Çünkü soru hemen şudur: hangi kitabın adı? Bir kitabın adı ve yazarı o kitaba özeldir. Oysa "ISBN nedir" sorusunun cevabı belirli bir kitaba özel değildir.

`Kitap.getAd()` yazınca "non-static method getAd() cannot be referenced from a static context" derleme hatası alınır (javac ile denendi). `kitap1.isbnNedir()` ise derlenir; `-Xlint:all` ile derlenince statik metodun nesne yerine sınıf adıyla çağrılması gerektiği uyarısı gelir (javac ile denendi).

Soru: Statik kullanmanın asıl amacı nesne yaratmadan doğrudan sınıftan çağırabilmek mi? Bunun yanında başka bir etkisi var mı?

Cevap: Toprak Hoca, değişkenlerin saklandığı yerle de ilgili olduğunu, statik üyenin belirli bir nesneye değil bütün sınıfa ait bilgiyi sakladığını söyledi. Ahmet Hoca iki kullanımı ayırarak ekledi:

- **Metot olarak:** Sık kullanılan, nesne üretilmesine gerek olmayan metotları statik tanımlamak daha kolaydır.
- **Alan (member) olarak:** En basit örnek, şu ana kadar sınıftan kaç nesne üretildiğini tutmaktır. Değişken her nesne oluştuğunda bir artırılır. Arka planda gizlice tutulabilir. Her

defasında baştan başlamaz, kaldığı yerden devam eder. Bellekte farklı bir yerde saklanır; bu yerleşimi hoca daha sonra anlatacağını söyledi.

2.7 Statik Sayaç

Bu açıklamanın ardından ikinci bir kitap oluşturuldu ve sayaç yazdırıldı. `getKitapSayisi` de statik olmalıdır; Insert Code ile üretilen metot zaten statikti. İki kitap oluşturulduğu için çıktı 2 olur (javac ile denendi).

2.8 Sınıfın Tam Kodu

```
public class Kitap {
    private String ad;
    private String yazar;
    private final String ISBN;          // bir kez atanır
    private static int kitapSayisi = 0; // bütün kitaplar için ortak

    public Kitap(String ad, String yazar, String ISBN) {
        this.ad = ad;
        this.yazar = yazar;
        this.ISBN = ISBN;
        kitapSayisi++;
    }

    public String getAd() { return ad; }
    public String getYazar() { return yazar; }
    public String getISBN() { return ISBN; }
    public static int getKitapSayisi() { return kitapSayisi; }
    public void setAd(String ad) { this.ad = ad; }
    public void setYazar(String yazar) { this.yazar = yazar; }

    public static String isbnNedir() {
        return "isbn kitaplari ayirt etmek icin essiz bir numaradir";
    }

    @Override
    public String toString() {
        return "Kitap(Ad=" + ad + ", Yazar=" + yazar
                + ", ISBN=" + ISBN + ")";
    }
}
```

3. Raf Sınıfı

Sıradaki sınıf `Raf`'tır. Tek bir alanı vardır: kitap dizisi. Metotları da basittir; diziye kitap eklenir, diziden kitap çıkarılır.

3.1 Alan ve Constructor

Az önce `private String` yazıyorduk, yani tipini yazıyorduk. Şimdi tip `Kitap` dizisidir: `private Kitap[] kitaplar`.

Constructor yine Insert Code ile üretilip değiştirildi. Otomatik üretilen constructor parametre alır, ama burada dışarıdan bir şey alınmayacak. İçeride dizi ilklendirilir: `this.kitaplar = new Kitap[10]`. Böylece her raf oluşturulduğunda 10 alanlı bir dizi oluşur.

Soru: `new Kitap[10]` yazınca oradaki `new`'in amacı nedir? Normalde `new` yeni nesne yaratır. Burada nesne dizisi mi yarattık?

Cevap: Evet. İçine kitap alabilen bir dizi yaratıldı. Bellekte, içinde 10 kitap bulunabilecek bir yer açıldı. `int` dizisi tanımlamak gibi düşünülebilir.

Ahmet Hoca bu ifadeyi 7. bölümde netleştirdi: dizide fiili kitap yoktur, 10 kitabın referansı için yer vardır.

3.2 kitapEkle

`public void kitapEkle(Kitap k)` metodu boş yer bulup kitabı oraya koyar. Başta bütün hücreler `null`'dır. Dizi doldukça `null` olmayan hücreler artar. O hâlde dizi gezilir, `null` olmayan hücreler geçilir, ilk `null` hücreye eklenir.

Gezme işi `while` ile yapıldı. Hoca önce `i`'yi `null` ile karşılaştıracak şekilde yazdı, sonra düzeltti: `null` olabilecek olan `i` değil, kitapların `i`. elemanıdır. Son hâli şudur:

1. `i` sıfırdan başlar.
2. `i < kitaplar.length` ve `kitaplar[i] != null` olduğu sürece `i` artar.
3. Döngü iki şarttan biriyle biter: boş yer bulundu ya da dizi bitti.
4. Hangi şarttan çıkıldığı kontrol edilir: `i != kitaplar.length` ise boş yer bulunmuştur ve `kitaplar[i] = k` atanır. Hoca burada `<` da kullanılabileceğini söyledi.

Hocanın özeti: kitap geldi, boş yer bulduk, kitabı koyduk.

> **Ek bilgi (derste anlatılmadı):** `while` şartındaki sıra önemlidir. `&&` soldaki şart yanlışsa sağdakine bakmaz; önce `i < kitaplar.length` yazıldığı için dizi dışına çıkılmaz. Şart ters yazılıp `kitaplar[i] != null && i < kitaplar.length` olunca, dolu 2 elemanlı bir dizide `Index 2 out of bounds for length 2` hatası alınır (javac ile denendi).

3.3 kitapSilSiraNo

İlk metot silinecek hücrenin sıra numarasını alır. Hoca önce sınır kontrolünün gerekçesini söyledi: bu metoda 20 de verilebilirdi, biz ise şu anda sabit 10 kullanıyoruz. Bu yüzden sıra numarası `kitaplar.length`'ten küçükse işlem yapılır: `kitaplar[siraNo] = null`.

Silme işlemi hücreyi `null` yapmaktır. İlk oluşturduğumuzda bütün hücreler `null`'dı; silince hücre yine o hâle döner.

> **Ders sonrası not:** Bu kontrol yalnız üst sınırı denetler. `kitapSilSiraNo(20)` hata vermez, ama `kitapSilSiraNo(-1)` çağrısı `ArrayIndexOutOfBoundsException` verir (javac ile denendi).

3.4 kitapSil

İkinci silme metodu `public void kitapSil(Kitap k)`'dır. Bu kez sıra bilinmez, kitabın kendisi verilir; kaçıncı sırada olduğunu bulmak gerekir.

Kod `kitapEkle`'ye çok benzer. Orada `null` aranıyordu; burada `k` aranır. `kitaplar[i] != k` olduğu sürece ilerlenir, `k` görülünce durulur. Sonra aynı kontrol yapılır, çünkü döngüden dizi bittiği için de çıkılmış olabilir. Kitap bulunduysa hücre `null` yapılır.

3.5 rafGoster ve getKitaplar

`public void rafGoster()` parametre almaz. `i` sıfırdan `kitaplar.length`'e kadar gider ve her hücre `System.out.println` ile yazdırılır. `getKitaplar` ise Insert Code ile üretildi.

3.6 Sınıfın Tam Kodu

```
public class Raf {
    private Kitap[] kitaplar;

    public Raf() {
        this.kitaplar = new Kitap[10];   // 10 boş (null) yer
    }

    public void kitapEkle(Kitap k) {
        int i = 0;
```

```
        while (i < kitaplar.length && kitaplar[i] != null)
            i++;
        if (i != kitaplar.length)        // boş yer bulunduysa
            kitaplar[i] = k;
    }

    public void kitapSilSiraNo(int siraNo) {
        if (siraNo < kitaplar.length)
            kitaplar[siraNo] = null;
    }

    public void kitapSil(Kitap k) {
        int i = 0;
        while (i < kitaplar.length && kitaplar[i] != k)
            i++;
        if (i != kitaplar.length)
            kitaplar[i] = null;
    }

    public void rafGoster() {
        for (int i = 0; i < kitaplar.length; i++)
            System.out.println(kitaplar[i]);   // boşsa "null" yazar
    }

    public Kitap[] getKitaplar() { return kitaplar; }
}
```

3.7 Rafın Testi

Hoca yanlış yapmış olabileceğimizi söyleyerek sınıfı test etti. Adımlar şöyledir:

1. `Raf raf1 = new Raf();` ile raf oluşturulur ve `raf1.rafGoster()` çağrılır. İçinde hiçbir şey olmadığı için 10 satır `null` basılır.
2. Üç kitap eklenir: `kitap1`, `kitap2`, `kitap3`. Üç kitap ve kalan yerlerde `null` beklenir, öyle de olur.
3. `kitap1`'in adı daha önce `Kitap2` yapıldığı için bu kitap da `Kitap2` görünür, ama yazarı `Yazar1`'dir. İkisi yazarlarından ayırt edilir.
4. Sıra numarasıyla silme denenir. 2 verilince ortadaki kitap değil, sonuncu gider. Ortadakini silmek için, indis sıfırdan başladığı için, 1 verilir.
5. `raf1.kitapSil(kitap3)` ile kitap verilerek silme denenir. Bir kitap kalması beklenir, fakat çıktıda silinmemiş görünür. Bir öğrenci sebebi söyler: silmeden sonra `rafGoster` çağrısı eklenmemiştir. Çağrı eklenince tek kitap kalır.

Bu sıranın çıktısı aşağıdadır (javac ile denendi). Hücre sayısı 10'dur; aşağıda kalan `null` satırları kısaltıldı. `ISBN2` ve `ISBN3`, altyazıda okunamayan değerlerin yerine konmuş yer tutucular-dır.

```
--- ekleme
Kitap(Ad=Kitap2, Yazar=Yazar1, ISBN=1234)
Kitap(Ad=Kitap2, Yazar=Yazar2, ISBN=ISBN2)
Kitap(Ad=Kitap3, Yazar=Yazar3, ISBN=ISBN3)
(7 satır null)
--- kitapSilSiraNo(1)
Kitap(Ad=Kitap2, Yazar=Yazar1, ISBN=1234)
null
Kitap(Ad=Kitap3, Yazar=Yazar3, ISBN=ISBN3)
(7 satır null)
--- kitapSil(kitap3)
Kitap(Ad=Kitap2, Yazar=Yazar1, ISBN=1234)
(9 satır null)
```

3.8 == ile Nesne Karşılaştırma

Soru: `kitapSil` içindeki karşılaştırma hep `false` vermez mi?

Cevap: Vermez. Burada nesnenin kendisi veriliyor. Toprak Hoca bunu `kitap1 == kitap1` örneğiyle gösterdi: aynı nesne olduğu için `true` dönmelidir. İki kitabın adı, yazarı ve numarası aynı olsa bile, farklı nesneler oldukları için `== true` döndürmez. İki kitabın aynı olup olmadığını test etmek istersek numaralarına bakmalıyız.

Ahmet Hoca karşılaştırmanın referans üzerinden yapıldığını ekledi. == iki nesne aynı referansı gösteriyorsa `true` verir. Aynı referans aynı yer demektir; dolayısıyla içerikler de aynıdır.

Aynı içerikli iki ayrı nesne için `a == b` sonucu `false`, ISBN'lerin `equals` ile karşılaştırılması `true` çıkar. `kitap1 == kitap1` ise `true`dur (javac ile denendi).

> **Ek bilgi (derste anlatılmadı):** İçerik karşılaştırmasını sınıfın kendisine yaptırmak için `Object`'ten gelen `equals` metodu ezilebilir. Ezilmemiş hâliyle, aynı içerikli iki `Kitap` için `a.equals(b)` sonucu `false` çıkar (javac ile denendi). Aşağıdaki ekleme `Kitap` sınıfına konunca aynı ISBN'li iki nesne için `== false`, `equals` `true` verir (javac ile denendi). `equals` tek başına ezilince `-Xlint:all` "overrides equals, but neither it nor any superclass overrides hashCode" uyarısını verdiği için `hashCode` da eklendi (javac ile denendi).

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Kitap)) return false;
    return ISBN.equals(((Kitap) o).ISBN);
}

@Override
public int hashCode() { return ISBN.hashCode(); }
```

4. Kitaplik Sınıfı

Son sınıf `Kitaplik`'tır. Alanı bir raf dizisidir: `private Raf[] raf`. Yapı iç içedir: kitaplıkta raf dizisi, her rafta kitap dizisi vardır.

4.1 Constructor: Her Hücre Ayrıca Üretilir

Bu constructor rafınkine göre biraz farklıdır. Raf 10'u sabit veriyordu; kitaplık kaç raf olacağını kullanıcıdan alır: `Kitaplik(int n)` içinde `this.raf = new Raf[n]`.

Ancak bu satır `n` elemanlı bir dizi oluşturur; hücrelerin içi dolu değildir. Her hücreye ayrıca bir raf üretilmelidir:

```
for (int i = 0; i < raf.length; i++)
    raf[i] = new Raf();
```

Hoca bunu yapmazsak `null` pointer hatası alacağımızı söyledi. Rafların tek tek ilklendirilmesi gerekir. Boş `Raf[2]` dizisinin ilk elemanına `rafGoster` çağrılınca `NullPointerException` alınır (javac ile denendi).

Rafların neden burada üretildiğini de kısaca söyledi: kitaplık yok edildiğinde raflar da yok olsun istiyorsak onları kitaplığın constructor'ında oluştururuz. Kitaplar ise dışarıda oluşturulup rafa eklendi; raf silinse de kitaplar var olmaya devam eder. Diyagramdaki iki farklı elmas bu farkı gösterir (7. bölüm).

4.2 Metotların Devri

Kitap ekleme ve silme metotlarının hepsi benzerdir. Hocanın anlatımı şöyledir: diyelim ki `Raf` sınıfını bir arkadaşımız yazdı. Biz burada yeni bir ekleme mantığı yazmıyoruz, işi o arkadaşın yazdığı metoda devrediyoruz.

- `kitapEkle(Kitap k, int rafNo)`: Hangi rafa ekleneceği alınır, `raf[rafNo].kitapEkle(k)` çağrılır.
- `kitapSilSiraNo(int rafNo, int siraNo)`: Hangi raftan ve hangi sıradan silineceği alınır. Hoca bir kitaplığı düşünmemizi istedi: rafları vardır, rafların da 10 sırası vardır. İki bilgi birlikte hücreyi belirtir.
- `kitapSil(int rafNo, Kitap k)`: İlgili rafa gidilir, kitap verilerek silinir.
- `rafGoster(int n)`: İlgili rafın `rafGoster` metodu çağrılır.
- `tumRaflariGoster()`: Raflar döngüyle gezilir. Her raftan önce kaçıncı raf olduğu belli olsun diye bir başlık yazdırılır, sonra aynı sınıfın `rafGoster(i)` metodu çağrılır.

4.3 kitapAra

Hoca, kitapçıya gidince genelde kitabın adını söylediğimizi hatırlatarak aramayı adla yaptı: parametre `String kitapAdi`.

Doğrusunun farklı olacağını da baştan söyledi: `Raf` sınıfına da bir `kitapAra` yazılmalı, kitaplıktan bu metot raf raf çağrılmalıdır. Bu kez bilerek öyle yapmadı. Amaç, diğer yolun ne kadar kolay olacağını görmektir.

Arama iki iç içe döngüyle yapılır:

1. `int n = raf.length;` alınır. Dış `for` `i`'yi 0'dan `n`'e kadar götürür ve rafları gezer.
2. İç döngü, rafın içini gezer. Burada gelişmiş `for` kullanıldı: `for (Kitap kitap : raf[i].getKitaplar())`.
3. Rafın dizisine `raf[i].kitaplar` diye doğrudan erişilemez, çünkü alan `private`'tır. Getter'ı bunun için yazdık.
4. String karşılaştırması yapılacağı ve hücrelerin çoğu `null` olduğu için önce `kitap != null` kontrol edilir. `null` ise o hücrede işlem yapılmaz; kontrol hata vermesin diye yazıldı.
5. Hücre doluysa `kitap.getAd()` ile ad alınır. Bu bir String'dir, iki String `equals` ile karşılaştırılır: `kitap.getAd().equals(kitapAdi)`.
6. Eşitse kitap bulunmuştur: hangi rafta olduğu yazdırılır ve `return kitap` ile kitap döndürülür.
7. Döngüler bittiyse kitap bulunamamıştır: `Kitap yok` yazdırılır.

Bu metot ilk eşleşmede çıkar. Hoca, raf düzeyinde bir aramada bulunduğu rafı söyleyip diğerlerinde de aramaya devam edilebileceğini de belirtti.

> **Ders sonrası not:** Altyazıda bulunamama dalında bir `return` söylenmiyor. `Kitap` döndüren bir metot döngüden sonra değer döndürmezse "missing return statement" derleme hatası alınır (javac ile denendi). Bu yüzden aşağıdaki kodda `return null;` vardır; kitap yoksa `Kitap yok` yazar ve metot `null` döndürür (javac ile denendi). Çağıran tarafın dönen değeri kontrol etmesi gerekir. Hoca "birazdan çağıracağız" demişti, ama altyazıda `kitapAra` çağrısı görülmüyor.

Toprak Hocanın tarif ettiği, derste yazılmayan yol aşağıdaki gibi olurdu:

> **Ek bilgi (derste anlatılmadı):** Hocanın daha doğru dediği tasarımda arama `Raf` sınıfına taşınır, `Kitaplik` yalnız raf raf çağırır. Kitaplık içinde iç döngü ve `null` kontrolü kalmaz. Bu metot, 1. rafa konan kitabı raf raf çağrılınca buldu (javac ile denendi).

```
// Raf sınıfında
public Kitap kitapAra(String ad) {
    for (Kitap k : kitaplar)
        if (k != null && k.getAd().equals(ad))
            return k;
    return null;
}
```

4.4 Metot Nesne Döndürebilir

Ahmet Hoca bu noktada araya girdi ve metodun `Kitap` türünden bir nesne döndürdüğüne dikkat çekti. Birçok alışveriş sitesinde bir şeyin adını yazarsınız, karşınıza sayfa sayısı, fiyat ve yazar gibi bilgiler gelir. Bu, böyle bir sorgudan dönen sonuçtur; veri ister veritabanından çekilsin ister başka yerden. Sorgu kimlik numarasıyla yapılıp kişinin bilgileri de dönebilirdi.

Bir de dizi döndürülebilir: bu kitap adında birden fazla kitap olabilir, o zaman şarta uyan kitapların dizisi döndürülür. Metodu çağırdığımız yerde dönen nesnenin bütün bilgilerine erişilir. Bunları `System.out.println` ile basabilir, isterseniz grafik arayüzde gösterebilirsiniz.

> **Hocanın vurgusu:** Dönüş değerine dikkat edin. Burada nesne arıyor ve nesne döndürüyoruz.

4.5 Soru: İç Döngü

Soru: İçteki döngüyü bir daha anlatabilir misiniz?

Cevap: Bu gelişmiş `for`'dur. Dönüş değişkeni `Kitap` tipinde `kitap` olur. `raf[i].getKitaplar()` bize bir raf dizisi verir; bu, raf oluşturulurken kurulan 10 elemanlı dizidir. Döngü bu 10 elemanın hepsinin üzerinden geçer. O an dolaşılan eleman `kitap` değişkenindedir.

> **Ek bilgi (derste anlatılmadı):** İç döngü, klasik `for` ile aşağıdaki gibi yazılabilir. Fark yalnız indisin görünmesidir. Bu biçim, içine tek kitap konan bir rafta aynı kitabı yazdırdı (javac ile denendi).

```
Kitap[] dizi = raf[i].getKitaplar();
for (int j = 0; j < dizi.length; j++) {
    Kitap kitap = dizi[j];
    // ...
}
```

4.6 Kapsülleme: İç Yapı Değişebilir

Toprak Hoca, `Kitaplik` metotlarının ne kadar basit olduğuna dikkat çekti. `Raf`'ı başka biri geliştiriyor olsaydı, `kitapEkle` ve `kitapSil`'in içinin nasıl olduğu umurumuzda olmazdı. Hatta burası bir veritabanı olsaydı ve bugün SQL kullanırken yarın başka bir veritabanına geçseydik, değişiklik yapılır, `Kitaplik`'ı ya da testi kullanan kişinin bundan haberi olmazdı. Aynı biçimde kullanmaya devam edilirdi.

`kitapAra` ise bunun tersini gösterir: işin çoğu `Kitaplik`'ta yapılıyor. Oysa bu iş rafa bırakılmalıydı.

4.7 Sınıfın Tam Kodu

```
public class Kitaplik {
    private Raf[] raf;

    public Kitaplik(int n) {
        this.raf = new Raf[n];
        for (int i = 0; i < raf.length; i++)
            raf[i] = new Raf();      // raflar kitaplığın içinde üretilir
    }

    public void kitapEkle(Kitap k, int rafNo) {
        raf[rafNo].kitapEkle(k);
    }

    public void kitapSilSiraNo(int rafNo, int siraNo) {
        raf[rafNo].kitapSilSiraNo(siraNo);
    }

    public void kitapSil(int rafNo, Kitap k) {
        raf[rafNo].kitapSil(k);
    }

    public void rafGoster(int n) {
        raf[n].rafGoster();
    }
```

```
    public void tumRaflariGoster() {
        for (int i = 0; i < raf.length; i++) {
            System.out.println(i + ". Raftaki Kitaplar");
            rafGoster(i);
        }
    }

    public Kitap kitapAra(String kitapAdi) {
        int n = raf.length;
        for (int i = 0; i < n; i++)
            for (Kitap kitap : raf[i].getKitaplar())
                if (kitap != null && kitap.getAd().equals(kitapAdi)) {
                    System.out.println(i + ". Rafta Kitap");
                    return kitap;
                }
        System.out.println("Kitap yok");
        return null;
    }
}
```

Ders sonrası not: Kısa nota göre UML'de metodun adı `gosterRaf(int)`, kodda `rafGoster(int)` olarak yazıldı. UML'e göre kodlarken adları birebir kullanmak gerekir.

5. Kitaplığın Testi

Eski test satırları yorum satırına alındı. Kitaplık `new Kitaplik(3)` ile üç raflı oluşturuldu. Kitap sayısı üçten altıya çıkarıldı.

```
public class Test {
    public static void main(String[] args) {
        Kitap kitap1 = new Kitap("Kitap1", "Yazar1", "ISBN1");
        Kitap kitap2 = new Kitap("Kitap2", "Yazar2", "ISBN2");
        Kitap kitap3 = new Kitap("Kitap3", "Yazar3", "ISBN3");
        Kitap kitap4 = new Kitap("Kitap4", "Yazar4", "ISBN4");
        Kitap kitap5 = new Kitap("Kitap5", "Yazar5", "ISBN5");
        Kitap kitap6 = new Kitap("Kitap6", "Yazar6", "ISBN6");

        Kitaplik kitaplik = new Kitaplik(3);
        kitaplik.kitapEkle(kitap1, 0);
        kitaplik.kitapEkle(kitap2, 0);
        kitaplik.kitapEkle(kitap3, 0);
        kitaplik.kitapEkle(kitap4, 1);
        kitaplik.kitapEkle(kitap5, 1);
        kitaplik.kitapEkle(kitap6, 2);

        kitaplik.kitapSilSiraNo(0, 1);   // 0. raftaki ortadaki kitap
        kitaplik.kitapSilSiraNo(1, 1);   // 1. raftaki 1 numaralı hücre
        kitaplik.tumRaflariGoster();
    }
}
```

Silmeden önce hoca kitap vererek silmeyi düşündü: 0. raftan altıncı kitap silinemez. Hata da vermez, mevcut kod yalnızca bir şey yapmaz. `kitapSil(0, kitap6)` çağrısı denenince hata çıkmadı ve raflar değişmedi (javac ile denendi). Bu yüzden sıra numarasıyla silmeye geçti.

Çalıştırmadan önce beklenti söylendi: 0. rafta ortadaki kitap gidecek. 1. rafta iki kitap var; sıra numarası sıfırdan başladığı için 2 numaralı eleman yok, 1 numaralı eleman gidecek. Sonuçta 1. rafta `Kitap4` kalır. Çıktı (javac ile denendi, kalan `null` satırları kısaltıldı):

```
0. Raftaki Kitaplar
Kitap(Ad=Kitap1, Yazar=Yazar1, ISBN=ISBN1)
```

```
null
Kitap(Ad=Kitap3, Yazar=Yazar3, ISBN=ISBN3)
(7 satır null)
1. Raftaki Kitaplar
Kitap(Ad=Kitap4, Yazar=Yazar4, ISBN=ISBN4)
(9 satır null)
2. Raftaki Kitaplar
Kitap(Ad=Kitap6, Yazar=Yazar6, ISBN=ISBN6)
(9 satır null)
```

Altı kitaptan ikisi silindiği için kitaplıkta dört kitap kalır: 0. rafta `Kitap1` ve `Kitap3`, 1. rafta `Kitap4`, 2. rafta `Kitap6` (python ile hesaplandı; aynı içerik Java çıktısında da var).

> **Ders sonrası not:** Kitaplıktaki metotlarda `rafNo` denetlenmiyor. Üç raflı kitaplıkta `kitapEkle(k, 5)` çağrısı `ArrayIndexOutOfBoundsException` verir (`Index 5 out of bounds for length 3`, javac ile denendi). Rafın kendisi de doluyken 11. kitabı sessizce almaz: on birinci `kitapEkle` hata vermez, kitap da eklenmez (javac ile denendi).

6. Tasarım Tartışması: Hangi Metot Hangi Sınıfa?

Ahmet Hoca, önemli olanın hangi sınıfların bulunduğu ve sınıflar arasındaki ilişkilerin nasıl kurulduğu olduğunu söyledi; veri yapıları dersindeki derinlikte algoritma bilgisi burada amaç değildir. Sonra öğrencilerden bu tasarıma ne eklemek istediklerini sordu. İki öneri geldi:

- **Rafa kitap dizisi ekleme:** Rafa tek bir kitap yerine bir kitap dizisi verilebilsin. Hoca bunu güzel bir özellik saydı. Bir önceki haftaki çalışmayla birleştirilip üzerinde çalışılabileceğini, dinamik diziler anlatıldıktan sonra bu konuya tekrar döneceklerini söyledi.
- **Silince kaydırma:** Bir kitap silinince arkasındaki kitaplar kaydırılsın. Statik dizi kullanıldığı için araya bir kaydırma (shift) işlemi konabilir. Hoca bunun da yapılabileceğini söyledi.

Sonra hoca bir soru sordu:

Soru: `Kitap` sınıfının içine `kitapEkle` diye bir metot yazsam makul olur mu?

Cevap: Mantıken doğru değildir. Kitabın ekleneceği yer ya raftır ya da kitaplıktır. Diziye kitap ekleme tarzındaki bir iş o sınıfların içinde olmalıdır.

> **Hocanın vurgusu:** Kurgudan hangi sınıfta hangi özelliklerin ve metotların olacağını çıkarabilmek lazım. Tasarım yaparken ilgisiz bir metodu ilgisiz bir yere yazmamak gerekir. Bu hemen olacak bir şey değil; örnek çözdükçe gelişir.

7. Bileşim ve Birleştirme

Bir öğrenci diyagramın büyütülmesini istemiş, hoca da elmasların anlamını açıklamaya geçmişti. Toprak Hoca, bu konuya ayrı bir şekil de getirmişti.

7.1 İki Elmas, İki İlişki Gücü

Ahmet Hoca iki sembolü şöyle ayırdı. Dolu olan elmas bileşimdir (composition), boş olan elmas birleştirmedir (aggregation). Fark ilişkinin gücündedir: koyu olan daha güçlü bir ilişkidir.

İki durumda da bir sahiplik vardır: raf kitaplara, kitaplık raflara sahiptir. Buna “has” ilişkisi diyoruz. Bu kavramlar, ilişkinin derecesini ve ciddiyetini anlatmak için ortaya çıkar.

- **Kitaplık ve raf (bileşim):** Kitaplık varsa raf da vardır. Bir kitaplıkta `n` raf olabilir ve aralarındaki varlık ilişkisi birbirine bağımlılık ifade eder. Hocanın benzetmesiyle: insan varsa kafası da vardır, insan yoksa kafasının da bir anlamı yoktur; tabii kafa nakli yapmadığımızı düşünüyoruz.
- **Raf ve kitap (birleştirme):** Raf varsa kitap olacak diye bir şey yoktur. Raf yoksa da kitap olabilir.

Toprak Hocanın şeklindeki örnekler de aynı ayrımı gösterir:

- **İnsan ve beyin (bileşim):** İnsan beyninden oluşur. İnsanı yok ettiğimizde beyinden bahsedemeyiz.
- **Araba ve yolcu (birleştirme):** Arabayı yok ettiğimizde hâlâ yolcudan bahsedebiliriz.

Toprak Hocanın aklında tutma yolu Türkçe karşılıklardır: bileşim "ondan oluşur", birleştirme "onu içerir".

7.2 Kodda Fark

Ahmet Hoca farkı doğrudan koda bakarak gösterdi. `Kitaplik` constructor'ındaki `raf[i] = new Raf()` satırında raf nesneleri fiilen oluşturuluyor. `Kitaplik` nesnesi silinirse, buradaki nesneler de onunla birlikte yok olur.

`Raf` constructor'ında ise durum farklıdır: `new Kitap[10]` yalnız 10 kitabı gösterebilecek referans için yer ayırır. Fiili bir kitap yoktur. Bir yerde fiilen raf nesnesi vardır, diğerinde yalnız referans vardır.

Toprak Hoca bunu kod kuralı olarak bağladı: koyu elmasta nesne sınıfın içinde (constructor'da) oluşturulur. Boş elmasta nesne dışarıda oluşur ve o nesne, onu içeren sınıf olmadan da var olmaya devam eder.

> **Hocanın vurgusu:** Raf nesnelerinin üretildiği satırla, referans dizisinin ayrıldığı satırın mantığını kavrarsanız iki ilişki arasındaki farkı anlarsınız. Bu iki kavram önemli olarak aklınızda kalsın; kodlaması da günlük hayattan örnekleri de var, kendi örneklerinizle zenginleştirin.

7.3 Nesneyi Yok Etmek Ne Demek?

Hoca, önceden yaptıkları bir örneği de anlattı. `Kitaplik` nesnesinden `getRaf` ile raf nesnesine erişilebildiğini düşünelim. Yeni bir kitaplık oluşturup eski referans değişkenine atayalım. Eski nesne boşa çıkar ve Java sanal makinesi onu siler. Silince raf nesneleri de tamamen yok olur, ama rafın tuttuğu kitaplar yaşamaya devam eder. Hoca bunu daha sonra istisnalar konusunda örnekleyeceğini söyledi.

> **Ders sonrası not:** Hoca "kitaplığı delete edersem" dedi; Java'da böyle bir komut yoktur, `delete o;` derlenmez (javac ile denendi). Nesne, referansı kalmayınca toplanabilir hâle gelir. Hocanın "raflar tamamen yok olur" cümlesi de koşulludur. `getRaf` eklenmiş bir kopyayla yapılan denemede, eski kitaplık ve dışarıdan referansı olmayan raf `System.gc()` sonrası toplandı; `getRaf` ile referansı dışarıda tutulan raf ise yaşamaya devam etti. Kitap da referansı `main`'de tutulduğu için yaşadı (javac ile denendi). Ayrıca kısa nottaki UML'de `getRaf` metodu yoktur.

8. Nesneler Üzerinde Yer Değiştirme Sorusu ve Sıradaki Konular

Bir öğrenci, günün dersinde geçen bir konu hakkında sordu.

Soru: Bir örnekte değerleri kalıcı olarak değiştirme gibi konular işlenecek mi, yoksa dersteki kadarı yeterli mi?

Cevap: Ahmet Hoca, ilerleyen haftalarda `ArrayList`'ler anlatılınca nesne sıralama (object sort) konusunun geleceğini söyledi. Orada işler nesneler üzerinden yürüyeceği için bir anlamda bu konuya da girilecek. İki kitap nesnesinin yer değiştirmesi gibi düşünülebilir. Bir sıralama yapılır ve konuya orada tekrar değinilir.

İlişki türlerinden association ve extend (kalıtım) da kaldı. Bunlar gelecek haftaların konularındadır. Dinamik diziler de rafa dizi ekleme önerisi için gereken konulardan biridir.

Dersin Toparlaması

Bu uygulamada, UML ile verilen üç sınıflı bir tasarım koda döküldü. `Kitap` sınıfında `private`, `final` ve `static` alanların farkı görüldü: `final` alanın setter'ı olmaz, statik sayaç bütün nesneler için ortaktır, statik metot nesne olmadan sınıf adıyla çağrılır. `toString` nesneyi anlamlı

yazdırır. `Raf` sınıfında nesne dizisi kuruldu, boş yer arama ve hücreyi `null` yaparak silme yazıldı. == karşılaştırmasının referans üzerinden yapıldığı görüldü. `Kitaplik` sınıfında işler rafa devredildi ve iç içe döngüyle `Kitap` döndüren bir arama yazıldı.

Dersin kavramsal sonucu iki noktadır. Birincisi, her metot ilgili olduğu sınıfa yazılmalıdır; kullanıcı iç yapıyı bilmeden sınıfı kullanabilir. İkincisi, bileşim ve birleştirme aynı "has" ilişkisinin iki gücüdür: kitaplık rafları kendi içinde üretir, raf ise dışarıda üretilmiş kitapların referansını tutar. Sonraki haftalarda association, kalıtım, `ArrayList` ve nesne sıralama gelecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.